

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 5](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 4: Divide and Conquer

Algorithms based on the *divide and conquer* technique reduce larger inputs into a many inputs which are b times smaller, solve them using the same algorithm, then combine them into the solution of the full problem. Binary search can be understood as an example of the *divide and conquer* technique—after comparing x with the number in the middle, it is sufficient to solve one problem (i.e., $a=1$) which is two times smaller (i.e., $b=2$)! We will see more examples in this lecture.

Example: Multiplication of Polynomials

In this example, we use arrays to denote polynomials. The array $a[0..n-1]$ is used to denote the polynomial

$$A(x) = \sum_{i=0}^{n-1} a[i]x^i;$$

similarly, the array b is used to denote $B(x)$ and the array c to denote $C(x)$. We want to multiply two such polynomials.

INPUT: Arrays $a[0..n-1]$, $b[0..n-1]$

OUTPUT: Array $c[0..2n-2]$ such that $C(x) = A(x)B(x)$.

For example, for $a=[1,1,1]$, $b=[1,2,2]$, we have $c=[1,3,5,4,2]$. To see it, observe that $A(x)=x^2+x+1$ and $B(x)=2x^2+2x+1$. Thus,

$$C(x) = A(x) \cdot B(x) = (x^2+x+1) \cdot (2x^2+2x+1) = 2x^4 + 4x^3 + 5x^2 + 3x + 1.$$

If you prefer, you can set $x=10$ and think about multiplication of numbers rather than polynomials. Thus, $A(10)$ is a number, and a contains its consecutive digits (starting with the least significant one). The example above corresponds to $111 \cdot 221 = 24531$. Numbers in a positional system are multiplied in the same way as polynomials with the only difference that one has to 'carry' when a temporary result is greater than 9.

In primary school, we already learned a trivial algorithm that simply multiplies each value in a with each value in b . This algorithm has the time complexity $\Theta(n^2)$.

The more efficient Karatsuba algorithm is based on divide and conquer. The idea is to reduce the input problem to three smaller problems each of half size. Assume that a and b have an even number of elements, that is n is even and can be written as $n=2s$ for some integer s . Split the arrays a and b into two halves of length s each. Thus, we obtain

$$\begin{aligned} a_0 &= a[0..s-1], \\ a_1 &= a[s..n-1], \\ b_0 &= b[0..s-1], \\ b_1 &= b[s..n-1]. \end{aligned}$$

Recall that a represents the polynomial $A(x)$ and b the polynomial $B(x)$. We have

$$A(x) = A_0(x) + x^s A_1(x),$$

and

$$B(x) = B_0(x) + x^s B_1(x).$$

Therefore,

$$C(x) = A(x) \cdot B(x) = A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^s + A_1(x)B_1(x)x^{2s}.$$

Define $(A_0+A_1)(x)$ as the polynomial obtained by adding the polynomials $A_0(x)$ and $A_1(x)$ together; similarly, define $(B_0+B_1)(x)$.

Let

$C0(x) = A0(x)B0(x),$
 $C1(x) = (A0 + A1)(x)(B0 + B1)(x),$
 and

$C2(x) = A1(x)B1(x);$

thus, we have

$C(x) = C0(x) + (C1(x) - C0(x) - C2(x))x^s + C2(x)x^{2s}.$

Note that the discussion above works also for odd n by taking s to be $n/2$ rounded down.

This leads to the following algorithm (pseudocode):

```

multiply(a[0..n-1], b[0..n-1]):
  if n=1:
    just multiply a[0] by b[0] and return their product
  else:
    let s=n/2    # integer division (n/2 rounded down)
    a0=a[0..s-1]
    a1=a[s..n-1]
    b0=b[0..s-1]
    b1=b[s..n-1]
    c0=multiply(a0,b0)
    c1=multiply(a0+a1,b0+b1)
    c2=multiply(a1,b1)
    return c0 + shift(c1-c0-c2, s) + shift(c2, 2s)
  
```

In this pseudocode, $+$ and $-$ correspond to addition/subtraction of polynomials, which can be done in $O(n)$ —for instance, for addition, simply return the array whose i -th element is $a[i] + b[i]$ for each i . The operation $\text{shift}(a, i)$ adds i zeros to the front of array a , which corresponds to multiplication of A by x^i .

What is the time complexity of this algorithm? Let us first assume that n is always even, that is, that n is of the form $2d$. Note that to multiply A and B of

length $n > 1$, we need to do three multiplications of numbers of length $s = n/2$ (here we use that n is even), as well as several additions, subtractions and shiftings, which are all done in $\Theta(n)$ time. Therefore, if we use $T(n)$ to denote the running time of this algorithm for arrays of length n , we get the equation

$$T(n) = 3T(n/2) + \Theta(n)$$

for $n \geq 2$ (and $T(1) = \Theta(1)$).

How to solve this formula? Recall that $n = 2^d$ for some integer d . By rewriting $T(n/2)$ using the same formula, and repeating until we reach $T(1)$, we obtain that

$$T(2^d) = \Theta(2^d) + \Theta(2^{d-1}) + \dots + \Theta(2^1) + \Theta(1) = \Theta(2^d).$$

(The last equality is not obvious but we omit the details.) Thus,

$$T(n) = \Theta(3 \log_2 n) = \Theta(n \log_2 3).$$

What is the running time if n is not a power of 2? In such a case, we pull a trick! If you are interested, continue to read! Take the largest $2^d \leq n$, set $n' = 2^d$ and observe that, by our choice, we have $n/2 \leq n' \leq n \leq 2n'$. As n' and $2n'$ are power of twos, our solution above gives us

$$T(n') = \Theta(n' \log_2 3) = \Omega((n/2) \log_2 3) = \Omega(n \log_2 3 \cdot (1/2) \log_2 3) = \Omega(n \log_2 3)$$

as well as

$$T(2n') = O((2n') \log_2 3) = O((4n) \log_2 3) = O(n \log_2 3 \cdot 4 \log_2 3) = O(n \log_2 3).$$

Since running time functions are nondecreasing, we have $T(n') \leq T(n) \leq T(2n')$, and therefore $T(n) = \Omega(T(n')) = \Omega(n \log_2 3)$ and $T(n) = O(T(2n')) = O(n \log_2 3)$. Thus, the total running time is $T(n) = \Theta(n \log_2 3)$!

Since $1.584 \dots < \log_2 3 < 1.585 \dots$, the Karatsuba algorithm based on divide and conquer runs in time $O(n^{1.585})$, which is better than the trivial algorithm that runs in time $\Theta(n^2)$. The memory complexity of both algorithms is $O(n)$ (for Karatsuba algorithm this will be shown below). Thus, for example, if we take $n = 1024$, we can expect the trivial algorithm to execute 1048576 (> 1 million) multiplications, while the Karatsuba algorithm will execute just 59049 ($< 60k$) multiplications—much faster! In practice, the hidden constant in the O notation is quite high, so the trivial algorithm will still run faster for small values of n . However, we can easily address this issue. Let K be the value of n when Karatsuba becomes faster. By changing the first line of our algorithm from "if $n = 1$ " to "if $n < K$ " and by using the trivial algorithm to solve this case, our algorithm becomes also fast for small values of n .

Note that the Karatsuba algorithm is not the best known algorithm for this problem—Fast Fourier Transform (FFT) can be used to multiply polynomials or large numbers in roughly $O(n \log n)$ (this algorithm will not be covered in this course).

Master Theorem for Divide-And-Conquer Recurrences

The [Master theorem for divide-and-conquer recurrences](#) provides a general way to solve recurrences like the one above.

Master Theorem

Suppose that

$$T(n) = aT(n/b) + f(n)$$

or

$$T(n) = aT(\lfloor n/b \rfloor) + f(n)$$

or

$$T(n) = aT(\lceil n/b \rceil) + f(n)$$

where a and b are constants. Let

$$c = \log_b a.$$

The following holds:

- **(Case A)** if $f(n) = O(n^{c'})$ for some $c' < c$, we have
 $T(n) = \Theta(n^c)$;
- **(Case B)** if $f(n) = \Theta(n^c)$, we have
 $T(n) = \Theta(n^c \log n)$;
- **(Case C)** if $f(n) = \Omega(n^{c'})$ for some $c' > c$, and $af(n/b) \leq kf(n)$ for some $k < 1$ (so-called *regularity condition*), then
 $T(n) = \Theta(f(n))$.

In the first case, the recursive part (solving the subproblems) dominates the non-recursive parts (splitting into subproblems and merging the results). In the last case (which rarely occurs in practice), the non-recursive part dominates. In the middle case, both the recursive and non-recursive part contribute to the resulting complexity.

Examples:

- $T(n)=3T(n/2)+\Theta(n)$ (Karatsuba algorithm) is an example of the first case (Case A),
- $T(n)=7T(n/2)+\Theta(n^2)$ is achieved by the Strassen algorithm for multiplying two matrices each of size $n \times n$; the algorithm could be understood as the matrix version of the Karatsuba algorithm. By solving the recursion (Case A), we get $T(n)=\Theta(n \log 27)=O(n \cdot 2.808)$, again with a rather big constant. Matrix multiplication is an important research problem—there are algorithms better than $\Theta(n \log 27)$, and we expect to find even better algorithms in the future.
- Binary search can be described with $T(n)=\Theta(1)+T(n/2)$, which is covered by the second case (Case B). Not surprisingly, we obtain $T(n)=\Theta(\log n)$.
- The merge sort algorithm (described in the next lecture) has the following recursive relation: $T(n)=\Theta(n)+2T(n/2)$, which is also covered by the second case (Case B). Thus, merge sort runs in time $T(n)=\Theta(n \log n)$.
- We could write a recurrence for the memory complexity of the Karatsuba algorithm: to multiply polynomials of size n , we use several auxiliary arrays of size n , and perform the multiplication of polynomials of size $n/2$. Since we run each of the three multiplications one after another, we can reuse the same memory. Thus, we get the recurrence $T(n)=\Theta(n)+T(n/2)$. This is covered by the last case (Case C), and we get that the memory complexity is $\Theta(n)$.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:18

[◀ Lecture 5 - recording](#)

Przejdź do...

[Lecture 6 & 7 ▶](#)

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553